ImpactKV: Coding-Aware Lossy KV Reuse for Shifted File-Island Prefill

Anonymous Author(s)

Abstract

Coding agents repeatedly ingest the same repository files across planner, implementer, tester, and reviewer turns, but those files rarely sit at a shared prompt prefix. Token IDs match; RoPE phases and surrounding context do not. Exact prefix caches therefore miss the reuse, while general lossy copiers are not told which coding spans are safe to approximate.

This paper presents **ImpactKV**, a coding-aware true-lossy KV reuse path for that setting. A template compiler admits version-valid single-file `repository_code` modules whose token IDs match and whose logical shift is nonzero; it does not estimate Attention. Source-side K is pre-rotated by the known position delta so the residual RoPE Δ at copy is zero; V is copied as-is. A mechanically invalid copy is fail-closed: the engine discards the island and Dense-recomputes the span; it never serves a partial or misaligned island. Ordinary prefix reuse and template-guided prefetch of those same spans exist as separate modules; prefetch miss degrades to the owned copy. Prefetch and ordinary prefix reuse are off in the headline campaign.

On a 24-task convenience sample of SWE-bench Verified trajectories (not the 500-task split; 235 rolling-6 target groups; 421 true-lossy islands) with Qwen2.5-Coder-7B-Instruct, ImpactKV copies 1684/1684 planned islands with zero fallback and is 1.492× cache-ready faster than the same SGLang Dense on that full PLAN (paired TTFT win rate 99.3%). A larger AWQ checkpoint is a different serving point and is not this headline: the same protocol on Qwen3-Coder 30B-A3B Instruct (AWQ 4-bit) is 1.375× cache-ready in the appendix. The evaluation is sequential one-token prefill versus own Dense, not concurrent load, P99 SLO, or session completion. One-token output agreement is 93.6% and is *not* official SWE-bench resolved. We do not claim a system-wide throughput crown.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → **Main memory**; • **Computing methodologies** → *Artificial intelligence.*

Keywords: KV cache, RoPE, prefill, multi-agent systems, code generation

1 Introduction

LLM serving engines treat the key–value (KV) cache as a first-class memory object: it is paged, shared, and evicted much like a process working set [5, 21]. That analogy breaks for multi-agent coding workloads. A later role re-reads a repository file that an earlier role already encoded, but the file no longer sits at the same prompt index. Token IDs match; the virtual addresses do not. Rotary position embeddings attach a phase R_p to every key [15], so a radix or prefix cache—which requires $\Delta = 0$ and a shared left context—cannot map the bytes. Copying K/V across a nonzero shift $\Delta = t - s$ is therefore *true lossy reuse*: exact on tokens, approximate on activations.

This is a memory-system problem, not a prompting trick. The engine must decide (i) which spans are eligible physical pages, (ii) how to translate their addresses, and (iii) what to do when translation fails. Section 2 reviews today's KV managers: exact prefix caches miss the shift, and generic lossy copiers are not told which coding spans are safe. Section 3 shows that the coding protocol already names those spans, and that the activation residual lives in source-time K/V formation rather than suffix-attention collapse. We drop 48 zero-shift islands at plan time so the headline cannot be credited to the prefix problem CacheWise already studies [16].

ImpactKV is that rule plus a fail-closed translator. Eligible units are version-valid, single-file `repository_code` modules whose token IDs match the target and whose logical shift is nonzero. Source-side K is pre-rotated by the known Δ so the residual at copy is zero; V is stored unchanged. A leased island is addressed by source-prefix hash, content hash, and Δ. Token-identical bytes formed under a different left context are not a legal mapping. A mechanically invalid copy (token-hash mismatch, incomplete coverage, or allocation failure) discards the island and Dense-recomputes it—the analogue of refusing to map a page that did not pass translation. Two further modules compose on the same admit set: lossless radix prefix reuse (capped before the shifted island) and template-guided prefetch from later-roles in the coding protocol, not remaining uses. Prefetch miss degrades to the owned copy; it does not Dense-recompute a valid island. In the headline campaign prefetch and ordinary prefix reuse stay off, so Table 5 cannot be credited to a radix hit.

We evaluate the copy kernel on frozen SWE-bench Verified trajectories [4]: a 24-task live-agent convenience sample (not the 500-task split), reconstructed into 235 exact-prompt target groups (rolling-6 turns, not 235 tasks) and 421 true-lossy islands, run as sequential one-token prefill on Qwen2.5-Coder-7B-Instruct in SGLang. The 1.492× headline is that full 235-group PLAN (705 pairs), not a further subsample of those groups. The kernel copies 1684/1684 planned islands with zero fallback and is 1.492× cache-ready faster than the same engine's Dense prefill (paired TTFT win rate 99.3%).

One-token output agreement is 93.6% and is *not* official SWE-bench resolved. The 30B AWQ replay of the same protocol lives in the appendix and is not mixed into Table 5; that appendix campaign is 1.375× cache-ready. We do not claim concurrent load, P99 SLO, session completion, or a system-wide throughput crown.

ImpactKV makes three contributions.

- **A file-module admit policy for true-lossy reuse.** Eligibility is token-ID equality, a valid file version, and nonzero shift—not path names, AST labels, or Attention scores. Radix LCP and file islands are disjoint; unconstrained copy then adds 194,624 tokens the gate refused (Section 3).
- **Page identity against mixed-prefix reuse.** A leased island is addressed by source-prefix hash, content hash, and Δ. Token-identical bytes formed under a different left context are different physical pages (Section 6).
- **Isolation from prefix cache.** The headline campaign turns ordinary prefix reuse off so a radix hit cannot take Table 5. On the same 235-group PLAN, prefix-only is 1.526× and dual is 2.120×; the file-module increment given prefix is 1.390× and is not mixed into that table (Section 8).

Source-side K pre-rotation is the copy implementation, not a third idea: CacheSlide, RedKnot, KVLink, and Notes-at-Prefill already reposition cached K under a known Δ [6, 11, 12, 17]. Mechanical fail-closed copy discards a hash, coverage, or allocation miss; zero observed fallbacks do not justify treating that check as a contribution. Prefetch (M3) is off in the headline campaign.

A separate 3B activation probe (Section 3.2) locates the residual on source-time K/V formation, not suffix-attention collapse. It is not 30B native attention, not the admit rule, and not a 7B or 30B TTFT number.

All speedups below are cache-ready target TTFT versus the same SGLang Dense unless we say otherwise: the source KV already exists, matching the cache-ready protocol used by CacheBlend and KVCOMM. Prefetch is disabled in the headline campaign. One-token replay is not official resolved.

2 Background: KV Management Today

Serving engines treat the KV cache as a first-class memory object: it is paged, shared, and evicted much like a process working set [5, 21]. Prefill time and GPU memory both grow with prompt length, so every reuse that avoids a Dense prefill is a systems win—if the mapping is legal.

2.1 Exact prefix reuse

When two requests share a byte-identical prefix at the same positions, a radix or paged prefix cache is exact and cheap. PagedAttention stores KV in pages and shares exact prefixes [5]. SGLang's radix tree does the same for structured programs [21]. LMCache makes that prefix portable across GPU, host, and remote backends [10]. KVFlow schedules and prefetches those prefixes for agent graphs [13]. CacheWise (ForeCache) is the closest coding-agent serving system: it cuts session completion by prefix-aware scheduling and eviction when Δ=0 [16].

All of these systems require $\Delta = 0$ and a shared left context. They are the right answer for that case. They are silent when a later role re-reads the same file at a new index.

2.2 Lossy copiers and lifetime managers

CacheBlend fuses cached knowledge into a new prefix by recomputing a fraction of tokens [18]. KVCOMM communicates almost the entire prompt across agents [19]. RelayCaching reuses decode-phase KV as later prefill [2]. These methods copy a lot and are typically fast. They are not told which coding spans are safe to approximate: a debugger note that names the candidate patch is the same kind of "context" as a version-valid repository file.

Continuum pins KV across tool calls with a TTL [7]. Tokencake schedules multi-agent jobs around the cache [1]. Retention and job scheduling are not a file-level admit rule.

RoPE attaches a phase R_p to every key [15]. Reusing a key computed at s in a target that needs t requires $R_t R_s^{-1} = R_\Delta$ on K; V and the surrounding attention context remain those of the source. CacheSlide, RedKnot, KVLink, and Notes-at-Prefill already apply that closed-form rotate under a known Δ [6, 11, 12, 17]. The rotate is the copy implementation, not a coding-file whitelist.

2.3 Limitation

Exact prefix caches miss shifted files. Generic lossy copiers ignore which files may be approximated. Lifetime managers pin or evict without asking whether the bytes are a version-valid `repository_code` module. Copying K/V across $\Delta = t - s \neq 0$ is therefore *true-lossy reuse*: exact on token IDs, approximate on activations, and currently unguided by the coding protocol. Section 3 asks whether that protocol supplies the missing admit rule.

3 Motivation: Coding Structure Can Guide KV

SWE-bench asks a model to resolve a GitHub issue against a real repository [4]. Multi-agent wrappers split planner, implementer, tester, and reviewer roles [9, 14]. Each role re-reads files that an earlier role already encoded, often after a tool observation that shifts later tokens. Figure 1 is that role graph. Trajectories come from a 24-task Verified live-agent run (rolling-6 history, 28K prompt cap). They are the evaluation substrate, not official Accuracy.

3.1 Complementary reuse the prefix cache cannot take

Figure 2 (top) measures the split on the 7B PLAN (job 137185 tokens, no new GPU). Every group has a nonempty radix

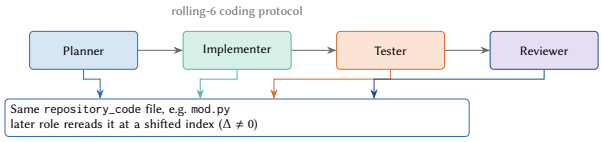

Figure 1. Coding roles reread the same file module at a shifted index.

LCP *and* a shifted file island. Mean LCP is 24.7% of the target prompt (1047 tok.); file-module copy is 33.4% (1537 tok.). The two token sets are disjoint (zero overlap, and LCP never reaches the first island). Prefix caches and CacheWise can take the left 24.7%; they cannot map the shifted 33.4%.

Unconstrained shifted copy does not stop at those file islands. Figure 2 (bottom) compares the 7B file-module PLAN with the KVCOMM-style PLAN from job 137400 (same target ids, no new GPU). The clone copies a campaign total of 194,624 extra tokens in 235/235 groups (spans the file gate refused). Almost every file-module token sits inside an unconstrained span (360,734 shared; 425 file-only). A general copier is faster because it moves those bytes; they are matching context, not version-valid repository files.

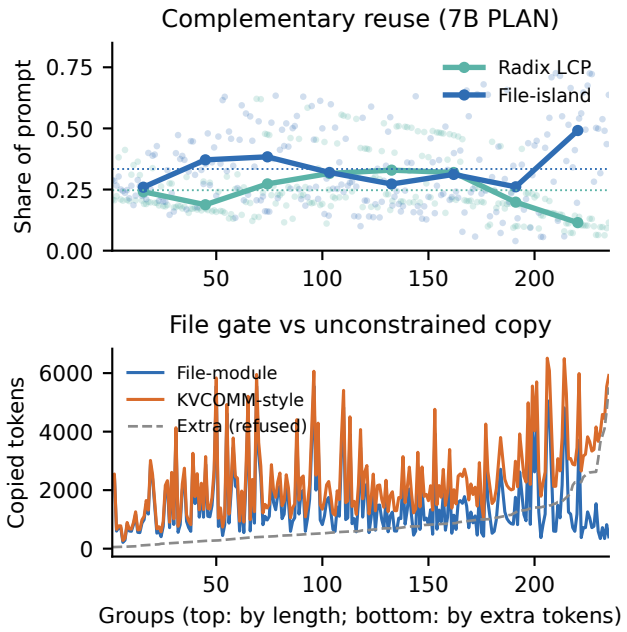

Figure 2. Top: complementary reuse on the 7B PLAN (235 groups). Mean LCP 24.7%; file-island 33.4%; overlap 0. Bottom: file-module versus unconstrained copy (job 137400 PLANs). Extra 194,624 tokens in 235/235 groups. Not a new GPU arm and not Table 5.

3.2 Where the loss sits

The 7B speed campaign in Section 8 does not record attention. A separate, frozen probe on Qwen2.5-Coder-3B [3] asks only where the activation error lives after a true-lossy file-island splice. It is not 30B native attention, not official resolved, and not the admit rule: the compiler in Section 5 still uses token-ID equality, not Attention scores.

Figure 3 and Figure 7 report that probe. TV here is the total-variation *distance between two attention distributions* (Dense versus splice), not a synonym for the KV tensors. On 26 single-island targets (13 tasks) that median attention TV is 0.00264 on the recomputed suffix and 0.00492 at the next-action marker, but 0.0462 on the copied island's *source-time formation* (about 4.4% of that formation mass has no counterpart in the target prefix). On 20 Dense prompts the last 32 query tokens put median 80.1% of attention on the top 10% of keys (within-observation top-20% mass 80.2%). An eight-prompt same-input diagnostic puts coding-aware module TV at 0.00214 against 0.0454 (CacheBlend) and 0.0745 (KVCOMM); that comparison is local TV, not matched-coverage Accuracy (Figure 4).

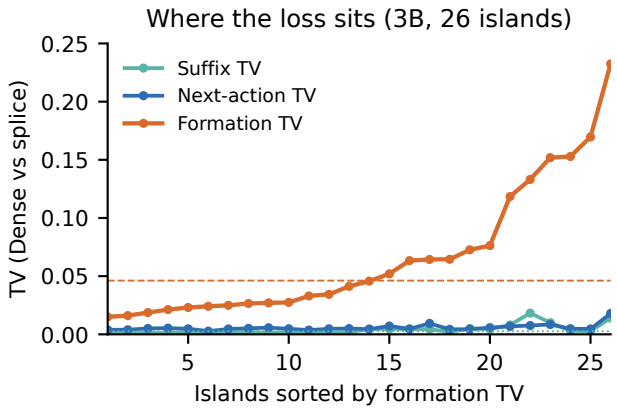

Figure 3. Per-island attention total-variation (TV) after a file-island splice (Qwen2.5-Coder-3B, 26 islands, sorted by formation TV). TV is attention-distribution distance, not a KV-cache quantity. Suffix attention does not collapse; formation TV is an order of magnitude larger (frozen medians 0.00264 / 0.00492 / 0.0462). Not 30B native and not Table 5.

The implication is directional, not a quality ranking: suffix attention does not collapse after a file-module copy, so a suffix-recompute copier is the wrong default repair. The larger drift is how the island's K/V formed under the source prefix. Unconstrained copiers additionally drift on tool and agent spans the file gate refused.

3.3 Opportunities

Three opportunities follow, and they are the design of Section 4.

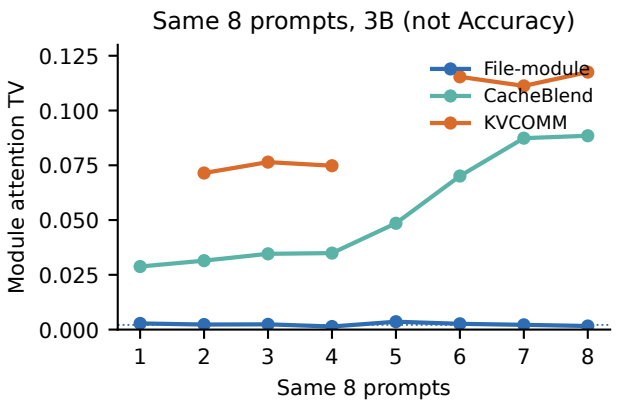

Figure 4. Same eight prompts, 3B: per-prompt module attention TV of file-module copy versus CacheBlend and KVCOMM. Local TV, not Accuracy and not 7B TTFT.

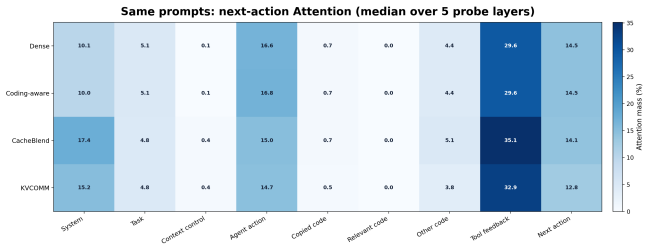

Figure 5. Next-action attention mass by prompt region, median over probe layers (same eight 3B prompts). File-module copy tracks Dense; CacheBlend and KVCOMM overweight system and tool-feedback tokens. Not Table 5.

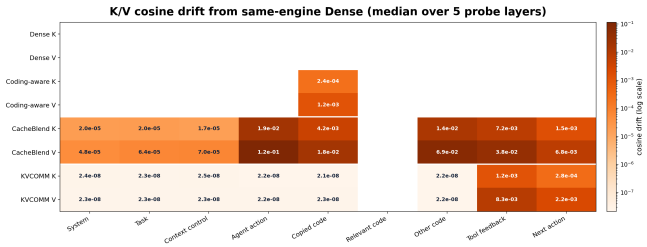

Figure 6. K/V cosine drift from same-engine Dense (same eight 3B prompts). File-module drift is confined to the copied code island; CacheBlend and KVCOMM drift on agent-action, other-code, and tool spans. Not 7B TTFT.

1. **Admit the shifted file, not the whole prompt.** 33.4% of the target is a version-valid `repository_code` island at $\Delta \neq 0$, disjoint from the 24.7% radix prefix. A coding-aware engine can copy that island and leave issue text, tool logs, and stale files Dense.
2. **Correct K phase; do not recompute the suffix.** Formation TV is ~17× suffix TV. Source-side K pre-rotation under the known Δ, with V copied as-is, matches where the residual actually lives.
3. **Fail closed on a mechanically invalid island.** Token-hash mismatch, incomplete coverage, or allocation failure must discard the whole island rather than splice a partial page. Unconstrained copy of the extra 194,624 tokens is faster and less agreed; it is not the policy.

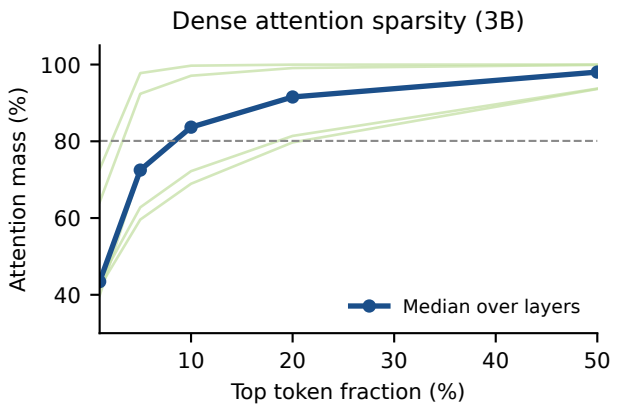

Figure 7. Frozen 3B Dense sparsity, not the 30B TTFT campaign. Median top-10% key mass 80.1% (n=20); within-observation top-20% mass 80.2%. Not official resolved.

4 Design

Section 3.3 asked for a file-module admit rule, a K pre-rotate rather than suffix recompute, and fail-closed copy. Figure 8 and Table 1 are that stack. Four modules compose the prefill path. The headline 7B campaign (job 137185) compiles M1 and M3 off so Table 5 cannot be credited to radix or prefetch. M0 does not estimate Attention. The rest of this section is the system model; Sections 5 and 6 instantiate M0 and M2.

4.1 High-level architecture

The online path is Dense prefix, then M2 copy of each admitted island, then Dense remainder. M1 and M3 stay off in the headline campaign so a radix hit or a prefetch cannot take Table 5. Figure 8 is that stack; Table 1 lists module I/O.

Table 1. Algorithm modules. Headline 7B speed uses only M0+M2. M3 miss degrades to M2; it does not Dense-recompute a still-owned island.

Module	Input	Output
M0 compiler	Frozen trajectories, tokenizer	PLAN: hashes, (s, t, L), $\Delta \neq 0$
M1 prefix	Source/target IDs, cap t	Radix LCP; no island copy
M2 copy	Admitted island, source K/V, Δ	Whole-island copy after K rotate, or Dense
M3 prefetch	Later-roles, residency, next-use	Device hints; miss → M2

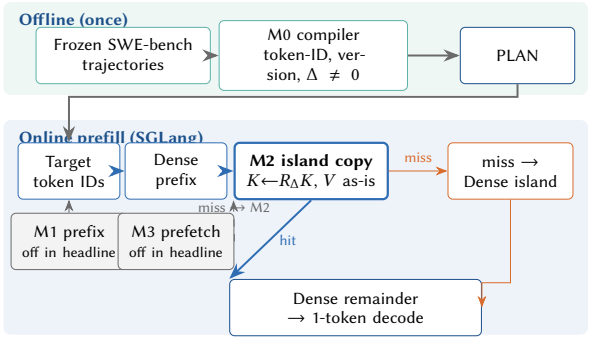

Figure 8. Engine stack for shifted file-module prefill. Online path is Dense prefix, then M2 copy, then Dense remainder. Headline 7B speed uses M0+M2; M1 and M3 stay off so Table 5 cannot be credited to radix or prefetch.

4.2 System model

ImpactKV targets multi-agent coding prefill in which:

- Each **task** is a repository-level issue: a natural-language specification, a codebase snapshot, and hidden tests that never enter the prompt.
- Each **turn** is one role request under a rolling-6 history (the last six complete role groups, compacted).
- A **file module** is a contiguous `repository_code` span for one path at one content version.
- The serving engine hosts Dense prefill and optional island copy on shared GPU memory.

A source request encodes a file at start index s. A later target request needs the same token IDs at start index t. Write $\Delta = t - s$. Ordinary prefix reuse applies only if $\Delta = 0$ and the bytes to the left also match. We study the complementary case $\Delta \neq 0$.

4.3 Design goals

1. **True lossy, not prefix in disguise.** Copied token IDs must match the target, and the logical shift is nonzero. Zero-shift islands are dropped so the method cannot degenerate to a prefix cache.
2. **Fail closed.** The admitted path pre-rotates source K so residual $\Delta = 0$ at copy. A mechanically invalid island is discarded and Dense-recomputed. Never serve a partial island.
3. **Cache-ready TTFT.** Target TTFT is reported after source KV already exists, against the same engine's Dense, matching CacheBlend and KVCOMM. Prefetch stays off in the headline 7B campaign. One-token agreement is not official resolved.

4.4 What would invalidate a claim

Mixing a 7B official-Accuracy table with the 7B or 30B TTFT numbers as if they were one method. Calling cache-ready 1.492× a one-use wall-clock serving win. Calling 93.6% one-token agreement Accuracy. Enabling prefetch or radix prefix reuse and attributing the gain to coding-aware copy. Editing the residual-Δ gate after seeing outcomes. Mixing the appendix 30B campaign's 1.375× into Table 5.

5 M0: File-Module Admit Compiler

5.1 Natural modules, not synthesized DAGs

The reusable objects are file modules already present in the agent protocol: roles read files, edit, test, and review under a rolling-6 history. We do *not* ask an LLM to invent a new DAG per task, and we do not treat planner-generated nodes as KV objects. Figure 1 is that role graph. The compiler is an offline oracle on frozen trajectories: it reconstructs each target prompt, locates `repository_code` spans, and keeps only islands that pass the eligibility gate (Figure 9). It is not an online serving policy.

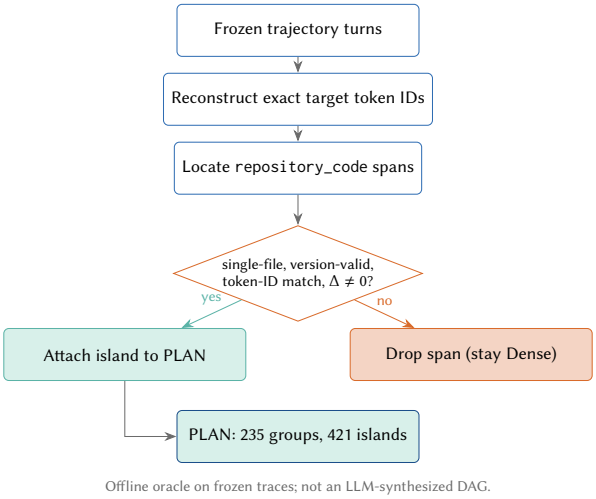

Figure 9. M0 offline compiler: frozen turns through the eligibility gate to a PLAN. Not an LLM-synthesized task DAG.

5.2 Eligibility gate

A span becomes a copy island only if every condition holds:

1. It is a single-file `repository_code` module (one path).
2. The content version is valid for that path in the trajectory (no stale-file copy after a later edit).
3. Source and target token IDs are identical on the span.
4. The logical shift $\Delta = t_{\text{start}} - s_{\text{start}}$ is nonzero. Zero-shift copies are dropped so the experiment cannot collapse to prefix reuse.

Algorithm 1 is this compiler.

On the frozen SWE-bench Verified traces this compiler yields 235 target groups and 421 islands, after dropping 48 zero-shift islands. Groups carry one, two, or three islands (89

Algorithm 1: Compile true-lossy file-module islands

Input: Frozen trajectories $\mathcal{T}$, tokenizer
Output: Target groups with true-lossy file islands

```
begin
    plan ← ∅
    for each target prompt P reconstructed from T do
        islands ← file modules of P
        for each candidate island I do
            if I is not single-file repository_code then
                continue
            if path version of I is stale then
                continue
            if token IDs(I) do not match a source span S then
                continue
            Δ ← I.start − S.start
            if Δ = 0 then
                drop I                    // not true lossy
            else
                attach I with source-side pre-rotate Δ
        if P has at least one attached island then
            plan ← plan ∪ {P}
    return plan
```

/ 106 / 40). Mean copied tokens per group are 1,537 against a mean target prompt of 4,433 tokens (median 4,239; p90 6,759; max 8,721). About one third of each target prompt is eligible true-lossy file content; the rest stays Dense.

5.3 What is stored, and how it is used

M0 is not a KV cache. It writes a PLAN of eligible islands (s, t, L, Δ); it does not allocate K/V. At serving time the source request Dense-prefills, and the controller stores only admitted islands under (*source-prefix hash*, *content hash*, Δ) with K already rotated (Section 6). The target request Dense-computes the prefix, copies each leased island into the target slots, Dense-computes the remainder, and decodes one token. Ineligible tokens—issue text, tool logs, stale files—never enter that store. Path and version metadata only locate the span; token-ID equality is the admit rule. The compiler does not estimate Attention and does not recompute KV distance online. Prefetch of those same spans (M3) uses later-roles in the coding protocol, not remaining target uses, and is off in the headline campaign.

6 M2: True-Lossy File-Island Copy

6.1 Islands as first-class KV objects

Each eligible file module is one physical island: a contiguous K/V range with a source start, a length, a target start, and a single Δ. Adjacent modules may share a source request but are not merged across different deltas. Ineligible tokens—system prompt, issue text, tool logs, debugger notes, and any file that failed the gate—stay on the Dense path.

The target compute path is therefore Dense prefix, then copy each admitted island, then Dense remainder, then decode. A leased island is either copied in full or discarded; there is no partial serve.

A leased island is a page, not a content hash. Its store identity is (*source-prefix hash*, *content hash*, Δ). Token-identical modules formed under a different left context, or pre-rotated by a different Δ, are different physical pages and must not share a handle. CacheWise maps Δ=0 prefixes; this key is what makes $\Delta \neq 0$ fail-closed rather than a silent mix of source-time K/V. Lookup-before-alloc may reuse a leased handle only when all three fields match.

6.2 Source-side K pre-rotation

After the source request, the controller copies selected K/V into controller-owned pool slots and leases them to the declared target uses. At materialization, K is rotated by Δ and V is stored unchanged:

$$K^{\text{pool}} = R_\Delta K^{\text{source}}, \qquad V^{\text{pool}} = V^{\text{source}}. \tag{1}$$

$R_\Delta = R_t R_s^{-1}$ is the closed-form RoPE correction. The copy applies the leftover $\Delta_{\text{res}} = (t - s) - \Delta_{\text{source}}$. The admitted campaign sets $\Delta_{\text{source}} = t - s$, so $\Delta_{\text{res}} = 0$ at copy and no extra rotate remains. Source-side pre-rotation is a speed optimization: it removes a per-target rotate from the copy critical path. It is not an Accuracy mechanism. V is never rotated, because RoPE does not act on values. The activation-level reason not to default to suffix recompute is Section 3.2: on Qwen2.5-Coder-3B the suffix attention TV is an order of magnitude smaller than the island's source-time formation TV. Pre-rotation corrects K phase under a known Δ; it does not estimate Attention and does not change the admit rule.

Figure 10 is the cross-turn mapping: an upstream read of a path becomes a downstream island at a new index.

6.3 Fail-closed residual check

Let $\Delta_{\text{res}} = (t - s) - \Delta_{\text{source}}$ be the remaining RoPE shift after the source-side rotate. The admitted path requires $\Delta_{\text{res}} = 0$:

$$\Delta_{\text{source}} = t - s \implies \Delta_{\text{res}} = 0 \quad \text{at copy.} \tag{2}$$

A copy that is not mechanically valid—token-hash mismatch, incomplete coverage, or allocation failure—is discarded and the span is Dense-recomputed. The engine never splices a partial island into the target cache. A leftover nonzero Δ_{res} is not the admitted fast path; the frozen campaign never takes it (421 source pre-rotations, 0 fallbacks). Zero observed

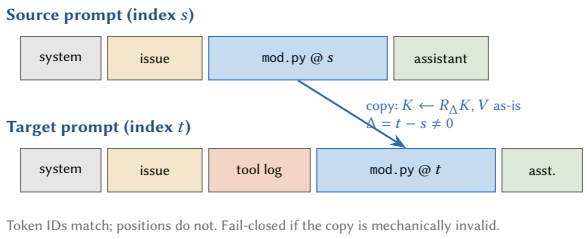

Figure 10. Cross-turn file-module reuse. Token IDs match; positions do not ($\Delta = t - s \neq 0$). Only K is phase-corrected; V is copied as-is.

fallbacks is not a reason to remove the check, and it is not a contribution. Prefetch miss on a still-owned island degrades to that owned copy; it does not Dense-recompute a valid island.

6.4 What stays off

Prefetch stays off in the headline 7B campaign, as does ordinary radix prefix reuse (M1 and M3 in Table 1). Host-overflow may hold source slots but is not a novelty claim. Copy tiling stays at the engine default (256 tokens). Those knobs are not Table 5.

Table 2 states the only policy the runtime consults.

7 Implementation

7.1 SGLang integration

ImpactKV is implemented in a SGLang fork as a controller-owned KV pool beside the ordinary request cache. The reuse arm publishes a dynamic manifest of sources and cases; the Dense arm uses the same generate path with copy disabled. Ordinary prefix reuse is compiled off in both arms so the contrast is island copy versus Dense prefill, not island copy versus radix.

Each source request writes selected K/V into controller-owned slots. Materialization applies `pre_rotate_delta` to K only, then records the applied delta in a per-source map. At the target the leftover rotate is $\Delta_{\text{res}} = (t_{\text{start}} - s_{\text{start}}) - \Delta_{\text{source}}$. The admitted campaign sets Δ_{source} to the logical shift, so $\Delta_{\text{res}} = 0$ and the copy path does not rotate again. Before a copy, the controller checks source token-ID hashes and full target coverage. Hash mismatch, an unowned gap, allocation failure, or a copy that is not mechanically valid raises a fallback event and the span is Dense-recomputed. The server ledger records copy events, fallback events, and the applied pre-rotate delta. Mechanical success is read from that ledger, not inferred from latency. Copy tiles stay at the engine default of 256 tokens.

Listing 1 is the conceptual manifest shape. Ordinary prefix reuse (M1) and prefetch (M3) are off in job 137185. A prefetch miss copies from the still-owned handle (M2) instead of Dense-recomputing the island.

Table 2. Admit policy for file modules. Anything else is Dense.

Object	Condition	Action
`repository_code`	IDs match, $\Delta \neq 0$, version OK	copy after K rotate
`repository_code`	$\Delta = 0$	drop (prefix)
`repository_code`	hash / coverage / alloc miss	discard, Dense
Issue / tool / role text	—	Dense
Stale or multi-file span	—	Dense

```
manifest = {
  "prefix_reuse": False, # M1 off
  "prefetch": False, # M3 off
  "sources": [{
    "id": "read:mod.py@v3", "start": 1842,
    "length": 1109, "pre_rotate_delta": -1250,
  }],
  "cases": [{
    "target_start": 592, "length": 1109,
    "source_id": "read:mod.py@v3",
  }],
}
```

Listing 1. Reuse manifest (conceptual)

7.2 Exact-prompt replay

The speed campaign does not re-sample the live agent. It reconstructs the exact target token IDs from the frozen trajectories with the same chat template and rolling-6 compaction, then replays those IDs. Decode is one token. Each group has one warmup and three measured rounds. The reuse arm materializes sources once per group; the Dense arm does not. Cache-ready TTFT is the target generate time after sources already exist, matching the protocol used by CacheBlend and KVCOMM.

Server processes restart every 20 groups to bound long-run memory drift. A group that fails mid-flight is retried after a fresh server. Those guards are operational; they are not part of the reuse policy.

7.3 Model and hardware

The headline model is Qwen2.5-Coder-7B-Instruct. The campaign ran as Slurm job 137185 on a compute GPU (not a debug or login node). Temperature is 0. The same protocol on Qwen3-Coder 30B-A3B Instruct (AWQ 4-bit, job 96092) is Appendix B; we do not mix that table with the 7B headline or with another model family's official-Accuracy table.

A PLAN group is hashes, (s, t, L), and $\Delta \neq 0$. Length, N-use, island-count, and repo slices are derived from the frozen `dense.json`/`reuse.json` pair; they are not a second GPU campaign. Porting to another tokenizer means decoding the island text and re-finding the span. Replaying 30B token ids on a 7B vocabulary is invalid.

8 Evaluation

8.1 Setup

Hardware and software. Engine: SGLang [21]. Model: Qwen2.5-Coder-7B-Instruct. Temperature 0, decode one token, one warmup and three measured rounds ($235 \times 3 = 705$ pairs). Slurm job 137185 on a compute GPU, not a login or debug node. Prefetch and ordinary prefix reuse are off. Appendix 30B uses Qwen3-Coder 30B-A3B Instruct (AWQ 4-bit, job 96092) on the same protocol. Replaying 30B token ids on 7B is invalid.

Dataset. SWE-bench Verified [4] trajectories from a 24-task live-agent run (expanded24). That is a convenience sample of Verified, not the full 500-task split. The 235 target groups are rolling-6 turns, not 235 tasks. Six repositories appear; 20 of 24 instances have at least one eligible $\Delta \neq 0$ group (Table 3). The speed campaign reconstructs exact target prompts and keeps only true-lossy file-module groups: 421 islands, 361,159 copied tokens per measured round, 48 zero-shift islands dropped at compile time (this is not a radix TTFT). There are 115 unique source prompts and 235 unique target prompts. Table 5 reports every group in that compiled PLAN (235 groups, 705 pairs). It is not a further subsample of the 235, and it is not a 500-task Verified run.

Baselines. Primary: the same SGLang Dense prefill on the same token IDs. Isolation on the same 235-group PLAN: prefix-only and dual prefix+copy (job 139839, Table 6). Admit clones: same-engine KVCOMM-style unconstrained copy and CacheBlend-style 15% boundary recompute (job 137400, Table 8), not native stacks. vLLM, LMCache, RelayCaching, Continuum, and Tokencake are different serving paths; this paper does not crown them.

Metrics. Cache-ready target TTFT versus the same engine's Dense: mean, median, p90, p99, and the ratio of means. Source KV already exists, matching CacheBlend and KVCOMM. Secondary: paired saving, paired win rate, copied-token fraction, mechanical copy and fallback counts, one-token output agreement. One-token agreement is *not* official SWE-bench resolved. We do not bill a source-inclusive N-use speedup.

8.2 Overall results

Table 5 and Figure 11 report the frozen 7B campaign (job 137185, COMPLETE). The same protocol on a larger AWQ checkpoint is a second serving point, not this table. Table 4 names both jobs; speedups stay in Table 5 and Table 10.

Table 4. Same file-module protocol at two checkpoints. Agreement and copy counts only; speedups stay in Table 5 and Table 10.

Checkpoint	Job	Agree	Copy
Qwen2.5-Coder-7B-Instruct	137185	93.6%	1684/1684
Qwen3-Coder 30B-A3B (AWQ)	96092	94.8%	1684/1684

Table 3. expanded24 dataset card. Groups are rolling-6 turns from 24 Verified tasks, derived from the 7B PLAN (job 137185) plus frozen trajectories, not a new GPU arm.

Item	Count
Verified tasks in producer	24
Repositories	6
Tasks with eligible groups	20
Target groups (turns)	235
True-lossy islands	421
Dropped Δ=0 islands	48

The copy kernel did what the policy asked: 421 source pre-rotations, 1684 copies over four rounds per island (one warmup plus three measured), and zero fallbacks. Mean cache-ready TTFT falls from 585.9 ms to 392.7 ms (1.492× ratio of means). The paired median is 555 ms versus 383 ms (27.9% saving); p90 is 952 ms versus 586 ms; p99 is 1178 ms versus 819 ms. ImpactKV is faster on 99.3% of the 705 pairs. Per-group speedup has median 1.389× and ranges 0.918×–3.369×. Groups with one, two, and three admitted islands are 89 / 106 / 40.

8.3 Ablation of each module

Job 139839 isolates M1 prefix reuse from M2 lossy copy on the 235-group 7B PLAN of Table 5, prefetch off. Speedups are versus this campaign's Dense, not Table 5. Figure 13 and Table 6 report the four arms. Prefix-only is 1.526× (940 nonempty prefix hits, median saving 32.9%, agreement 100%). Lossy-only is 1.408× (1684 copies, median saving 23.7%, agreement 93.6%). Dual is 2.120× (median saving 55.4%, win rate 100%, agreement 94.9%). The file-module increment given prefix is 1.390× (dual / prefix-only) and is not mixed into Table 5. Dual records 1126 copy events with 0 fallback. One-token agreement is not Accuracy.

Job 137400 runs two policy clones on the same 7B prompts, Dense, and SGLang executor as Table 5. KVCOMM-style admit copies unconstrained $\Delta \neq 0$ exact spans (no file-module gate). CacheBlend-style repair then Dense-recomputes 15% of each such span at the fusion boundary. Neither arm is the native CacheBlend or KVCOMM serving stack. Table 8 is not Table 5, not 30B, and not the seven-group dual-island table.

Figure 14 and Table 8 are that campaign. Unconstrained copy is faster because it moves more tokens: KVCOMM-style 2.100× cache-ready and CacheBlend-style 1.883×, versus file-module 1.492×. Median paired saving is 27.9% / 50.9% / 45.6%; win rates are 99.3% / 99.9% / 100%. One-token agreement falls from 93.6% (660/705) to 89.4% (630/705) and 91.9% (648/705). On 51 pairs the file-module output matches Dense while

Table 5. Qwen2.5-Coder-7B-Instruct exact-prompt true-lossy file-module replay (job 137185). Cache-ready TTFT is target generate time after source KV already exists. One-token agreement is not official resolved. Not the seven-group dual-island table and not the appendix 30B table.

Metric	Dense	ImpactKV
Target groups	235	235
Measured pairs	705	705
Mean prompt tokens	4433	4433
Copied tokens (mean)	—	1537 (33%)
Islands	—	421
Copy events / planned	—	1684/1684
Fallback events	—	0
K pre-rotations	—	421
Mean TTFT	585.9 ms	392.7 ms
Median TTFT	555 ms	383 ms
P90 TTFT	952 ms	586 ms
P99 TTFT	1178 ms	819 ms
Cache-ready speedup	1.000×	1.492×
Paired saving (mean / med.)	—	30.6% / 27.9%
Paired win rate	—	99.3%
One-token agreement	—	93.6%
Prefetch / prefix reuse	off	off

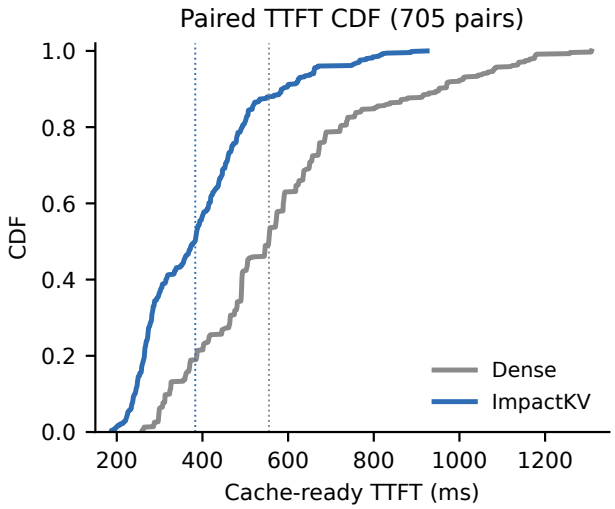

Figure 11. Empirical CDF of paired cache-ready TTFT (705 pairs, job 137185). ImpactKV shifts the body and the tail: median 555→383 ms, p90 952→586 ms, p99 1178→819 ms.

KVCOMM-style does not; the reverse is 21 pairs. Those 51 pairs are 17 groups with all three measured rounds. Table 7 decodes the extra tokens the file gate refused: tool observations, tool commands, and assistant turns wrapped around the file island, not a second admitted file module. On those 17 groups every extra token abuts a file island (campaign-wide

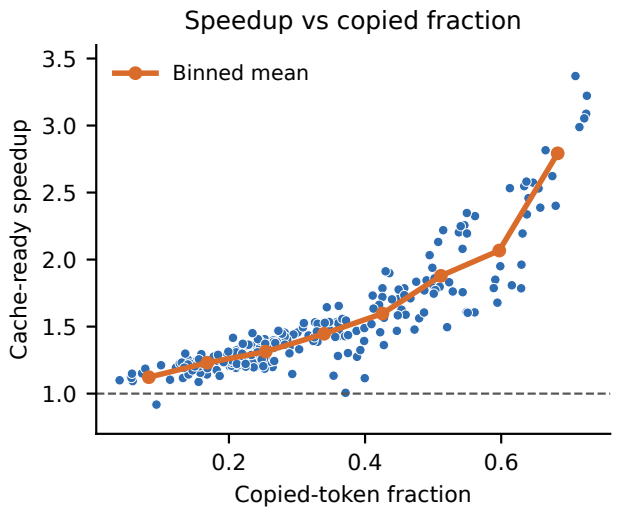

Figure 12. Per-group cache-ready speedup versus copied-token fraction, with a binned-mean line. Groups that copy more of the prompt are faster; the headline 1.492× is not a short-prompt artifact.

Table 6. Qwen2.5-Coder-7B-Instruct prefix-on increment on the 235-group PLAN of job 137185 (job 139839). Speedups versus this campaign's Dense. Prefetch off. Not Table 5, not the seven-group dual-island table, not 30B.

Arm	TTFT	Med. save	Win	Agree	Prefix	Copy
Dense	1.000×	—	—	—	—	—
prefix-only	1.526×	32.9%	99.9%	100%	940	0
lossy-only	1.408×	23.7%	98.9%	93.6%	0	1684
dual	2.120×	55.4%	100%	94.9%	940	1126

191,906/194,624). Figure 2 is the token-level reason: a campaign total of 194,624 extra unconstrained tokens, present in 235/235 groups. That is why the headline stays coding-aware file-module copy versus Dense, not a race on raw TTFT. We do not run a same-token 30B CacheBlend or KVCOMM arm. A greedy longest-span copier on the 30B ids (job 132385) is *not a comparison target*: agreement falls from 94.8% to 87.1%.

8.4 Sensitivity

Eligible islands are not tiny. Copied fraction grows with reconstructed prompt length: 29% below 3K tokens, 32% in 3–5K, 34% in 5–7K, and 50% at 7K and above (Table 13). Position deltas are strictly negative on this set (median $|\Delta| = 902$, range $[82, 7097]$): the same file appears earlier in the target than in the source, which is exactly the rolling-history pattern prefix caches miss.

Figure 15 splits the same 705 pairs four ways, derived from job 137185, not a new GPU arm. Numeric companions are Tables 14, 15, and 16 in the appendix. Speedup grows

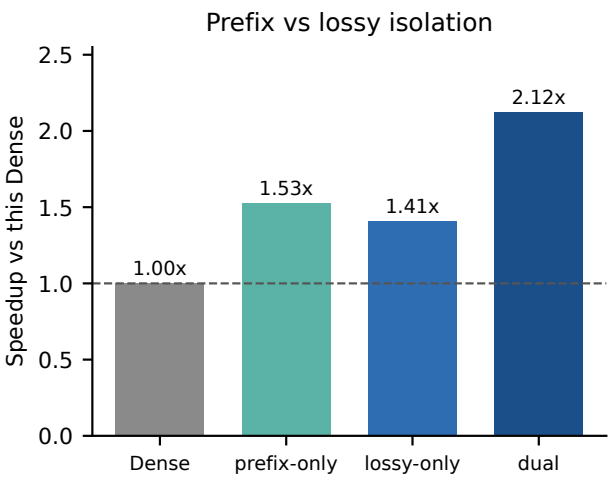

Figure 13. Prefix-on isolation on the 235-group PLAN (job 139839). Dual is copy plus radix, not a prefetch win. Not Table 5.

Table 7. Decoded KVCOMM-only extra tokens (job 137400 PLANs). Leftover prose is assistant; code dumps inside tool XML count as tool observations. Not a new GPU arm, not admitted `repository_code`, and not Table 5.

Kind	Campaign	17 groups (51 pairs)
Tool observation	117,965	8,156
Tool command	44,274	2,517
Assistant turn	28,471	1,723
Chat marker	3,914	243
Extra tokens	194,624	12,639
Abut a file island	191,906	12,639

Table 8. Admit ablation: same-engine policy clones on Qwen2.5-Coder-7B-Instruct SWE-bench prompts (job 137400). Not native CacheBlend/KVCOMM stacks. Speedups versus this campaign's 7B Dense. Not Table 5, not 30B, not the seven-group dual-island table.

Arm	TTFT	Med. save	Win	Agree	Copy
Dense	1.000×	—	—	—	—
File-module	1.492×	27.9%	99.3%	93.6%	1684/1684
KVCOMM-style	2.100×	50.9%	99.9%	89.4%	948/948
CacheBlend-style	1.883×	45.6%	100%	91.9%	948/948

with prompt length: 1.28× below 3K tokens, 1.43× in 3–5K, 1.50× in 5–7K, and 1.97× at 7K and above. It also grows with admitted files (1.315× / 1.492× / 1.833× for 1 / 2 / 3 islands) and with copied-token fraction (Q1 1.196×, Q4 2.116×). |Δ| is not the driver: a shift of 3000 or more is still 1.609×. Longer coding prompts and more copied file bytes, not quiz-length prefill, are where the copy pays.

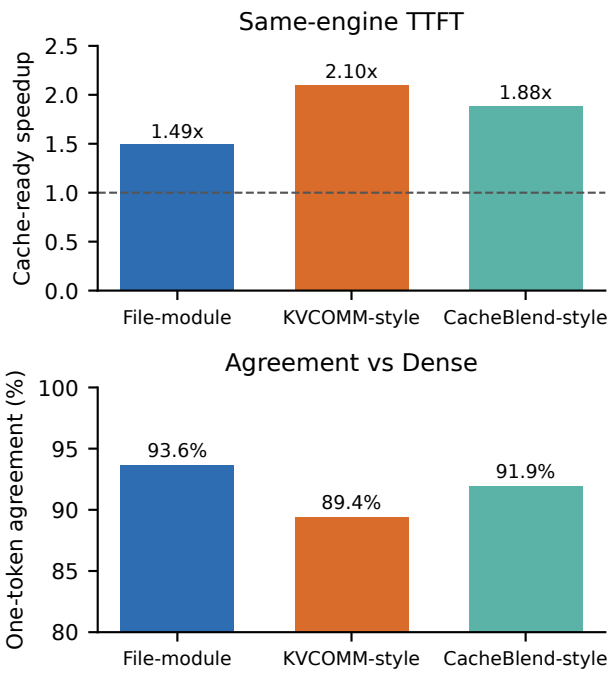

Figure 14. Same-engine admit clones (job 137400). Unconstrained copy is faster and agrees less. Not native CacheBlend/KVCOMM stacks and not Table 5.

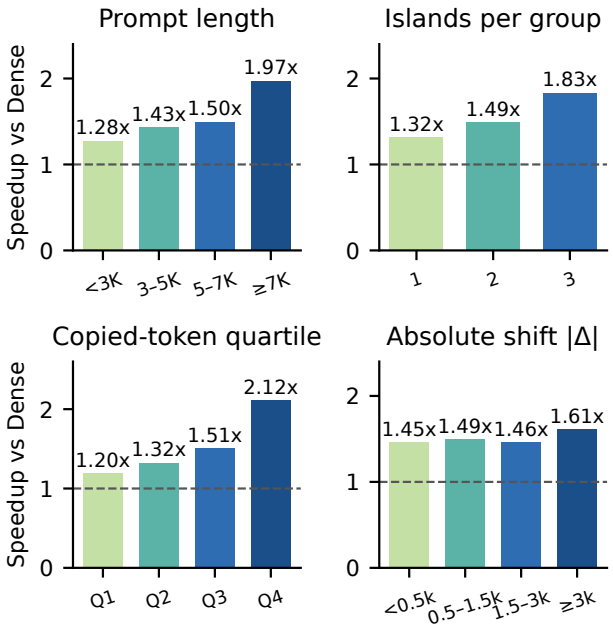

Figure 15. Cache-ready TTFT speedup on the same 705 pairs (job 137185). Not a new GPU arm and not Table 5.

Of 115 unique source prompts, 101 appear in more than one target group and 50 appear in five groups. That is why

the page key includes prefix and Δ, not content hash alone. Forty-eight zero-shift islands were dropped at plan time so this campaign cannot be credited to prefix cache.

8.5 Output agreement is not Accuracy

On the 705 measured pairs, 93.6% of one-token outputs match Dense. The artifact labels this field `not_accuracy`. Official SWE-bench resolved remains a live-agent, full-decode, hidden-test outcome. The source expanded24 live-agent run recorded Dense 3/24 and policy 5/24 resolved (McNemar exact two-sided $p = 0.625$). That contrast is the trajectory producer, not this replay, and it is not statistically significant. This paper does not claim an official Accuracy win. A content-only store on the same 7B PLAN mixed source-prefix pages (job 137091) and agreement was 66.4%; Table 5 is the prefix-keyed campaign.

The 7B PLAN has 105 unique content hashes; 92 appear in more than one group, and all 92 mix Δ inside the same 20-group restart window (7 also mix source prefix). That is why the page key includes prefix and Δ. Replaying 30B token ids on 7B is invalid. The seven-group dual-island isolation is Appendix A; Table 11 stays job 119795.

9 Related Work

9.1 Multi-agent systems for software engineering

Surveys of LLM agents for software engineering document planner–coder–tester decompositions, tool loops, and SWE-bench as the standard repository-level benchmark [4, 9, 14]. Most of that literature treats the serving engine as a black box. ImpactKV takes the opposite cut: freeze the agent protocol and ask which file modules the engine may copy.

9.2 Exact KV management

Section 2 reviews PagedAttention, SGLang radix, LMCache, KVFlow, and CacheWise (ForeCache). ContiguousKV aligns layout for prefill offload [22]. Those systems are the right answer when $\Delta = 0$. They do not copy a file module whose token IDs match at a nonzero shift. CacheWise's headline is agent *session completion* under prefix thrashing; ours is sequential one-token prefill of $\Delta \neq 0$ file islands.

9.3 Lossy and cross-agent reuse

CacheBlend recomputes a fraction of tokens when cached context is fused into a new prefix [18]. KVCOMM communicates KV across agents for unmatched context [19]. RelayCaching reuses decode-phase KV as later prefill [2]. These methods copy more and are typically faster. They are not a coding-file whitelist, and this paper does not claim a system-wide throughput crown against the native CacheBlend or KVCOMM serving stacks. On the 7B SWE-bench prompts we instead compare *same-engine policy clones*: unconstrained shifted-span copy (KVCOMM-style admit, no file gate) and a 15% boundary recompute on those spans (CacheBlend-style repair). Those clones share the SGLang executor and the job 137185 prompts; they are not the authors' CUDA kernels (Table 8). An eight-prompt same-input 3B diagnostic (Section 3.2) reports coding-aware module TV 0.00214 against 0.0454 (CacheBlend) and 0.0745 (KVCOMM); that is local TV, not a ranking and not matched-coverage Accuracy.

9.4 Agent scheduling and TTL

Continuum pins KV across tool calls with a latency-aware TTL [7]. Tokencake schedules multi-agent jobs around the cache [1]. Parrot exposes semantic variables to the serving runtime [8]. Plan caching stores agent plans [20]. ImpactKV is orthogonal: it decides which shifted file modules may enter a lossy copy after source-side K pre-rotation, and fail-closes on a mechanically invalid island. Prefetch is a residency side campaign (Table 11), not the headline copy kernel. TTL is out of scope.

9.5 Position-transformed KV reuse

RoPE supplies an explicit K phase correction [15]. CacheSlide reuses relative-position-dependent chunks under a known shift [11]. RedKnot applies that closed-form rotate inside a serving stack [12]. KVLink adjusts positional embeddings when concatenating independently precomputed document KV [17]. Notes-at-Prefill RoPE-repositions a precompiled skill and splices it [6]. ImpactKV does not claim that correction as novelty. It uses the same closed-form rotate as the copy implementation, and differs in *which* spans may enter the lossy path: version-valid single-file islands from an offline compiler, with page identity that forbids mixed prefixes. Prefetch stays off in the headline campaign.

10 Discussion and Limitations

The speed campaign is an exact-prompt, one-token replay of frozen trajectories. That is the right experiment for a copy kernel. It is the wrong experiment for claiming that coding-aware reuse resolves more GitHub issues. Official resolved on the 24-task live-agent producer is underpowered (McNemar $p = 0.625$) and is not this paper's headline.

Cache-ready 1.492× is real on 705 paired 7B target prefills. That number is target generate time after source KV already exists, not a cold-start wall-clock. The 30B analogue is reported only in the appendix: cache-ready 1.375×.

We do not mix this 30B long-prompt TTFT table with a different model family's official-Accuracy table. Doing so would manufacture a single official system that was never run. We likewise do not bind Figure 7 to the 30B cache-ready 1.375×.

Threats. Replay uses one token of decode, so TTFT is prefill-dominated. The 24-task producer is a convenience sample, not a powered Accuracy cohort. We do not run, and do not owe, a 500-task Verified replay, a concurrent P99 study, or a same-token 30B bake-off against CacheBlend/KVCOMM.

We do not replay all of SWE-bench Verified. Table 5 keeps prefix off; the prefix-on increment is Table 6. We do not treat Table 11 as the cache-ready 1.492× method. We do not full-decode the 17 copier-disagreement groups, and we do not run a 7B activation probe. The PLAN is an offline oracle over frozen trajectories, not an online admit service.

11 Conclusion

Coding agents re-read repository files at shifted positions. Prefix caches miss that reuse; generic lossy copiers ignore which files are safe. ImpactKV copies only version-valid, token-identical file islands with a nonzero shift, pre-rotates K on the source, and Dense-recomputes a mechanically invalid island. A leftover nonzero residual Δ is not the admitted fast path.

On 235 exact-prompt groups and 421 islands, the kernel copies 1684/1684 planned islands with zero fallback and is 1.492× cache-ready faster than the same SGLang Dense. Prefetch stays off. Sequential template prefetch matches copy (Table 11) and is not a speedup. Prefix+copy versus Dense is not a prefetch win. One-token agreement is not official resolved. We do not claim a system-wide throughput crown.

A Seven-group dual-island isolation

Qwen2.5-Coder-7B-Instruct on seven retokenized dual-island groups (jobs 124825–124829, COMPLETE) is not the 30B SWE-bench PLAN and is not mixed into Table 5. Prefix-only keeps radix Δ=0 and leaves the shifted island dense. Lossy-only copies the shifted file-island with prefix reuse off. Dual runs both. Template prefetch on this workload is not a speedup (combined vs dual 0.528×, 7 hints, 4 misses that then Dense-recomputed). That miss path is a module bug: M3 must degrade to M2, not disable copy.

B 30B AWQ replay

The headline uses Qwen2.5-Coder-7B-Instruct (job 137185). The same file-module protocol on Qwen3-Coder 30B-A3B Instruct (AWQ 4-bit, job 96092, COMPLETE) copies 1684/1684 islands with zero fallback and is 1.375× cache-ready (one-token 94.8%). Table 10 is that campaign. It is not mixed into Table 5. Prefetch (job 119795) is also 30B and is a residency check, not a 30B speed claim (Table 11).

Job 119795 is a four-mode campaign on the same 235 groups (705 pairs). It is not job 96092. The served path compiles next-island / later-roles hints, not remaining uses. Sequential already-resident next use has no overlap window, so spill and hint are skipped: DEVICE plus the immediate next target is not a prefetch win against copy that already holds the island on device. Combined cache-ready TTFT matches coding (1.392× vs 1.390× versus this campaign's own Dense): that is overhead, not a prefetch speedup. Prefetch-only is 0.996× and is not a copy win. A prefetch miss degrades to the owned copy; it must not Dense-recompute a valid island.

Table 9. Qwen2.5-Coder-7B-Instruct dual-island isolation. Speedups are versus this 7B Dense (21 pairs). Not the 30B PLAN and not Table 5.

Arm	vs Dense	Prefix hits	Copies	Fallback
Dense	1.000×	—	—	—
prefix-only	1.155×	28	0	0
lossy-only	4.526×	0	28/28	0
dual	8.490×	28	28/28	0

Table 10. 30B AWQ exact-prompt file-module replay (job 96092). Appendix only. Cache-ready TTFT excludes source/build. Not Table 5.

Metric	Dense	ImpactKV
Target groups	235	235
Copy events / planned	—	1684/1684
Fallback events	—	0
Cache-ready TTFT speedup	1.000×	1.375×
One-token output agreement	—	94.8%
Prefetch / prefix reuse	off	off

Table 11 is not mixed into Table 5. Dual prefix+copy versus Dense is not a prefetch number.

M3 compiles later-roles / next-island hints on already-admitted keys (Figure 16). It does not grow the copy set and does not rotate K. A sequential already-resident next use is skipped (`no_overlap_window`). Prefetch miss must keep the owned M2 handle (job 124825–124829 combined vs dual 0.528× was four misses that Dense-recomputed valid islands).

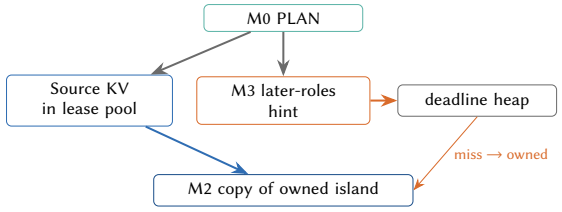

Figure 16. M3 residency hints on the M0 admit set. Miss degrades to the owned copy. Off in Table 5.

C expanded24 instances

Job 96092 reconstructs prompts from 24 SWE-bench Verified tasks. Groups are rolling-6 turns. Twenty instances have at least one eligible $\Delta \neq 0$ group: astropy-7166, 7606, 13236, 14539; django-13128, 13449, 15629; pydata xarray-4687, 7233; scikit-learn-11578, 14053, 14983, 26323; sphinx-8459, 9258, 9320, 9591; sympy-16597, 17139, 18199. Four producer tasks have no eligible group after the $\Delta \neq 0$ gate: django-13837, pytest-5631, sympy-14711, xarray-4629. Repo group counts

Table 11. Template-guided prefetch on sequential one-token replay. Combined is copy plus residency; prefetch-only is not a copy win. Speedups are versus this campaign's Dense, not Table 5. Hints are next-island / later-roles, not remaining uses. 30B appendix, not the 7B headline.

Arm	Speedup	Copy	Spill	Hints	Miss
Dense	1.000×	—	—	—	—
coding (copy)	1.390×	1684/1684	0	0	0
prefetch-only	0.996×	0	0	0	0
combined	1.392×	1684/1684	0	0	0

Algorithm 2: Compile next-island prefetch hints (M3)

Input: Observed PREFIX/MIDDLE islands with protocol later-roles
Output: Prefetch hints; copy set unchanged
for *each observation o* **do**
  if *o fails the M0 gate* **then**
    continue
  if *o is DEVICE and the next use is this request* **then**
    continue
  if *o is MIDDLE and later-roles* ≤ 0 **then**
    continue
  emit hint on *o*.key with priority = later-roles

(astropy 23, django 58, pydata 33, scikit-learn 33, sphinx-doc 51, sympy 37) sum to 235. This list is a convenience sample, not a Verified leaderboard.

Table 12 reports cache-ready speedup by repository from the same 705 pairs. Group counts are small; we do not rank repositories. pydata is the weakest slice (1.119×, 33 groups); django is the strongest (1.551×, 58 groups). Derived from job 96092, not a new GPU arm.

D 7B sensitivity tables

Numeric companions of Figure 15, derived from job 137185. Not a new GPU arm and not Table 5.

E Additional Figures

These figures expand the compile path and object lifecycle. The 11-page body is self-contained without them.

Table 12. Cache-ready TTFT by repository, derived from job 96092. Not a ranking and not Table 5. Appendix 30B slice.

Repo	Groups	Pairs	vs Dense
astropy	23	69	1.233×
django	58	174	1.551×
pydata	33	99	1.119×
scikit-learn	33	99	1.328×
sphinx-doc	51	153	1.435×
sympy	37	111	1.422×

Table 13. Target-prompt shape after the eligibility gate. Copied tokens are the true-lossy file-module payload, not a prefix.

Length	Groups	Mean tok.	Copied	Frac.
< 3K	42	2656	766	29%
3–5K	129	4086	1316	32%
5–7K	46	5757	1950	34%
≥ 7K	18	7677	3859	50%
All	235	4433	1537	33%

Table 14. Cache-ready TTFT by islands per target group, derived from job 137185 paired measurements. Not a new GPU arm and not Table 5.

Islands	Groups	Pairs	vs Dense
1	89	267	1.315×
2	106	318	1.492×
3	40	120	1.833×

Table 15. Cache-ready TTFT by absolute island shift, derived from job 137185. Not a new GPU arm.

$\lvert\Delta\rvert$	Groups	Pairs	vs Dense
< 500	41	123	1.455×
500–1500	97	291	1.489×
1500–3000	69	207	1.462×
≥ 3000	28	84	1.609×

Table 16. Cache-ready TTFT by copied-token fraction quartile, derived from job 137185. Not a new GPU arm.

Quartile	Groups	Pairs	vs Dense
Q1 (least copied)	59	177	1.196×
Q2	59	177	1.323×
Q3	59	177	1.508×
Q4 (most copied)	58	174	2.116×

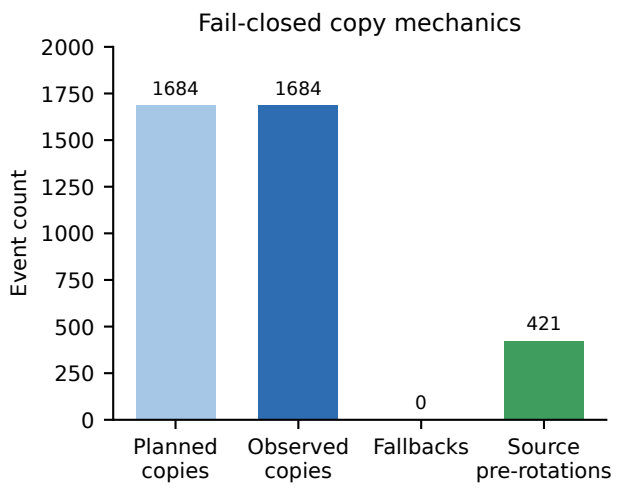

Figure 17. Fail-closed copy mechanics (job 137185). 1684/1684 planned copies, 0 fallbacks, 421 source K pre-rotations. Not mixed into Table 5.

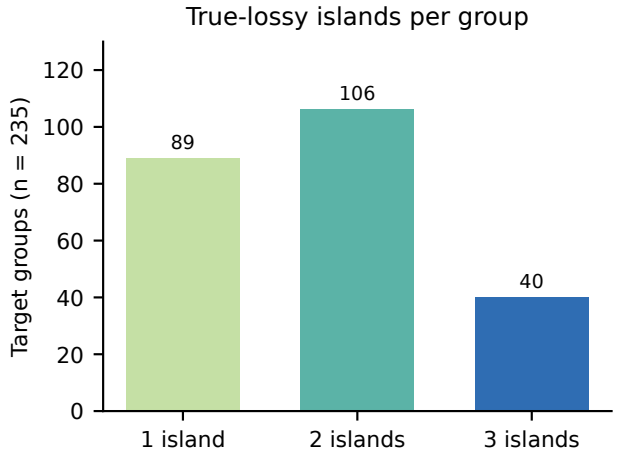

Figure 18. True-lossy islands per target group on the 7B PLAN (n=235). Not a new GPU arm and not Table 5.

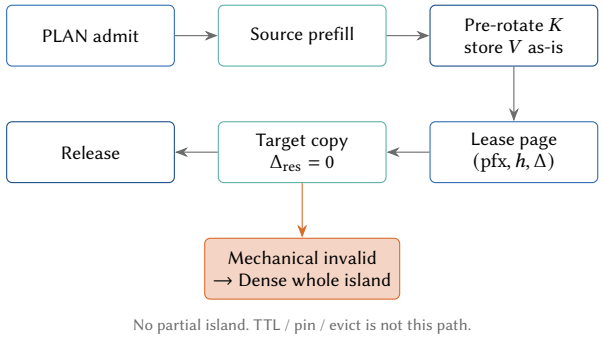

Figure 19. KV object lifecycle. A leased island is either copied in full or discarded; there is no partial serve.

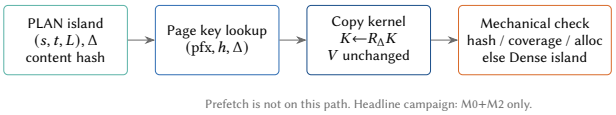

Figure 20. Compile path from a frozen target group to a leased island. Prefetch is not on this path.

References

[1] Zhuohang Bian, Feiyang Wu, Teng Ma, and Youwei Zhuo. 2025. Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. arXiv:2510.18586 [cs.DC] https://arxiv.org/abs/2510.18586

[2] Yingsheng Geng, Yuchong Gao, Weihong Wu, Guyue Liu, and Jiang Liu. 2026. RelayCaching: Accelerating LLM Collaboration via Decoding KV Cache Reuse. arXiv:2603.13289 [cs.LG] https://arxiv.org/abs/2603.13289

[3] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] https://arxiv.org/abs/2409.12186

[4] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VTF8yNQM66

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626. doi:10.1145/3600006.3613165

[6] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. arXiv:2606.17107 [cs.LG] https://arxiv.org/abs/2606.17107

[7] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2026. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS] https://arxiv.org/abs/2511.02230

[8] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 929–945. https://www.usenix.org/conference/osdi24/presentation/lin-chaofan

[9] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2025. Large Language Model-Based Agents for Software Engineering: A Survey. arXiv:2409.02977 [cs.SE] https://arxiv.org/abs/2409.02977

[10] Yuhan Liu et al. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665 [cs.DC] https://arxiv.org/abs/2510.09665

[11] Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. 2026. CacheSlide: Unlocking Cross Position-Aware KV Cache Reuse for Accelerating LLM Serving. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. https://www.usenix.org/conference/fast26/presentation/liu-yang

[12] Yang Liu, Zhaokai Luo, Huayi Jin, Ruozhou He, Chenchen Hong, Zhiyong Wang, Boyu Wang, Guanjie Chen, Tao Xie, and Junhao Hu. 2026. RedKnot: Efficient Long-Context LLM Serving with Head-Aware KV Reuse and SegPagedAttention. arXiv:2606.06256 [cs.AI] https://arxiv.org/abs/2606.06256

[13] Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, et al. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. *arXiv preprint arXiv:2507.07400* (2025).

[14] Zeeshan Rasheeda, Muhammad Waseema, Kai-Kristian Kemella, Mika Saari, and Pekka Abrahamsson. 2026. LLM-Based Multi-Agent Systems for Code Generation: A Multi-Vocal Literature Review. arXiv:2604.16321 [cs.SE] https://arxiv.org/abs/2604.16321

[15] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864* (2021).

[16] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. 2026. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents. arXiv:2606.16824 [cs.DC] https://arxiv.org/abs/2606.16824

[17] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. In *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2502.16002

[18] Jiayi Yao, Hanchen Li, Yuhan Liu, Sarah Ray, Yiying Cheng, Angela Zhang, Dong Du, Ion Stoica, and Hao Zhang. 2024. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. *arXiv preprint arXiv:2405.16444* (2024).

[19] Hancheng Ye, Tianyu Gao, Wenxuan Xu, et al. 2025. KVCOMM: Online Cross-context KV-cache Communication for Efficient LLM-based Multi-agent Systems. *arXiv preprint* (2025).

[20] Qizheng Zhang, Michael Wornow, Gerry Wan, and Kunle Olukotun. 2026. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. arXiv:2506.14852 [cs.DC] https://arxiv.org/abs/2506.14852

[21] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[22] Jing Zou, Shangyu Wu, Hancong Duan, Qiao Li, and Chun Jason Xue. 2026. ContiguousKV: Accelerating LLM Prefill with Granularity-Aligned KV Cache Management. arXiv:2601.13631 [cs.OS] https://arxiv.org/abs/2601.13631